



C Recursion

Last Updated : 15 Mar, 2024

In C programming language, you may have heard of the concept of **recursion**. You may also have heard that recursion is difficult and complex to understand and implement. Do not worry! In this article, we are going to cover the basics of recursion in C, recursive functions, recursive calls, and how it is different from iteration.



[DSA Course](#) [DSA](#) [Interview Problems on Recursion](#) [Practice Recursion](#) [MCQs on Recursion](#) [Recursion Tutorial](#) [Recursive Function](#) [Recur](#)



What is Recursion in C?

First, let's start with the recursion definition,



Recursion is the process of a function calling itself repeatedly till the given condition is satisfied. A function that calls itself directly or indirectly is called a recursive function and such kind of function calls are called recursive calls.

In C, recursion is used to solve complex problems by breaking them down into simpler sub-problems. We can solve large numbers of problems using recursion in C. For example, factorial of a number, generating Fibonacci series, generating subsets, etc.

Let's discuss some basic terminologies and fundamentals of recursion before going into working and implementation.

Recursive Functions in C

In C, a function that calls itself is called Recursive Function. The recursive functions contain a call to themselves somewhere in the function body. Moreover, such functions can contain multiple recursive calls.

Basic Structure of Recursive Functions

The basic syntax structure of the recursive functions is:

```
type function_name (args) {  
    .... // function statements  
    .... // base condition  
    .... // recursion case (recursive call)  
}
```

```
type function_name (args) {  
    // function statements  
  
    // base condition  
  
    // recursion case (recursive call)  
  
}
```

Basic Structure of Recursive Function in C

Example: C Program to Implement Recursion

In the below C program, recursion is used to calculate the sum of the first N natural numbers.

C



```
// C Program to calculate the sum of first N natural numbers  
// using recursion  
#include <stdio.h>  
  
int nSum(int n)  
{  
    // base condition to terminate the recursion when N = 0  
    if (n == 0) {  
        return 0;  
    }  
}
```

```
..... // recursive case / recursive call
..... int res = n + nSum(n - 1);

..... return res;
}

int main()
{
..... int n = 5;

..... // calling the function
..... int sum = nSum(n);

..... printf("Sum of First %d Natural Numbers: %d", n, sum);
..... return 0;
}
```

Output

Sum of First 5 Natural Numbers: 15

We will understand the different concepts of recursion using this example.

Fundamentals of C Recursion

The fundamental of recursion consists of two objects which are essential for any recursive function. These are:

1. Recursion Case
2. Base Condition

```
int nSum(int n)
```

```
{
```

```
    if (n==0) {  
        return 0;
```

Base
condition

```
    }
```

```
    int res = n + nsum(n-1);
```

```
    return res;
```

Recursive
case

```
}
```

Base Condition and Recursion Case for nSum() Function

1. Recursion Case

The recursion case refers to the **recursive call** present in the recursive function. It decides what type of recursion will occur and how the problem will be divided into smaller subproblems.

The **recursion case defined in the nSum()** function of the above example is:

```
int res = n + nSum(n - 1);
```

The recursion case is often represented mathematically as a **recurrence relation**. For the above case:

```
f(N) = N + f(N - 1);
```

This recurrence relation is used for the complexity analysis of the method.

2. Base Condition

The base condition specifies when the recursion is going to terminate. It is the condition that determines the exit point of the recursion.

***Note:** It is important to define the base condition before the recursive case otherwise, the base condition may never be encountered and recursion might continue till infinity.*

Considering the above example again, the **base condition defined for the nSum() function:**

```
if (n == 0) {  
    return 0;  
}
```

Now that the basic terminologies and fundamentals are out of the way, let's move on to understand how the recursion works in C.

How Recursion works in C?

Recursion is considered difficult to understand by many people but once you understand the working of recursion, it becomes a powerful weapon in your arsenal to battle complex problems.

To understand how C recursion works, we will again refer to the example above and trace the flow of the program. In the `nSum()` function, **Recursive Case** is

```
int res = n + nSum(n - 1);
```

In the example, $n = 5$, so as **`nSum(5)`**'s recursive case, we get

```
int res = 5 + nSum(4);
```

In **`nSum(4)`**, the recursion case and everything else will be the same, but $n = 4$. Let's evaluate the recursive case for $n = 4$,

```
int res = 4 + nSum(3);
```

Similarly, for **`nSum(3)`**, **`nSum(2)`** and **`nSum(1)`**

```
int res = 3 + nSum(2); ... // nSum(3)
int res = 2 + nSum(1); ... // nSum(2)
int res = 1 + nSum(0); ... // nSum(1)
```

Let's not evaluate nSum(0) and further for now.

Now recall that the **return value** of the nSum() function in this same integer named **res**. So, instead of the function, we can put the value returned by these functions. As such, for nSum(5), we get

```
int res = 5 + 4 + nSum(3);
```

Similarly, putting return values of nSum() for every n, we get

```
int res = 5 + 4 + 3 + 2 + 1 + nSum(0);
```

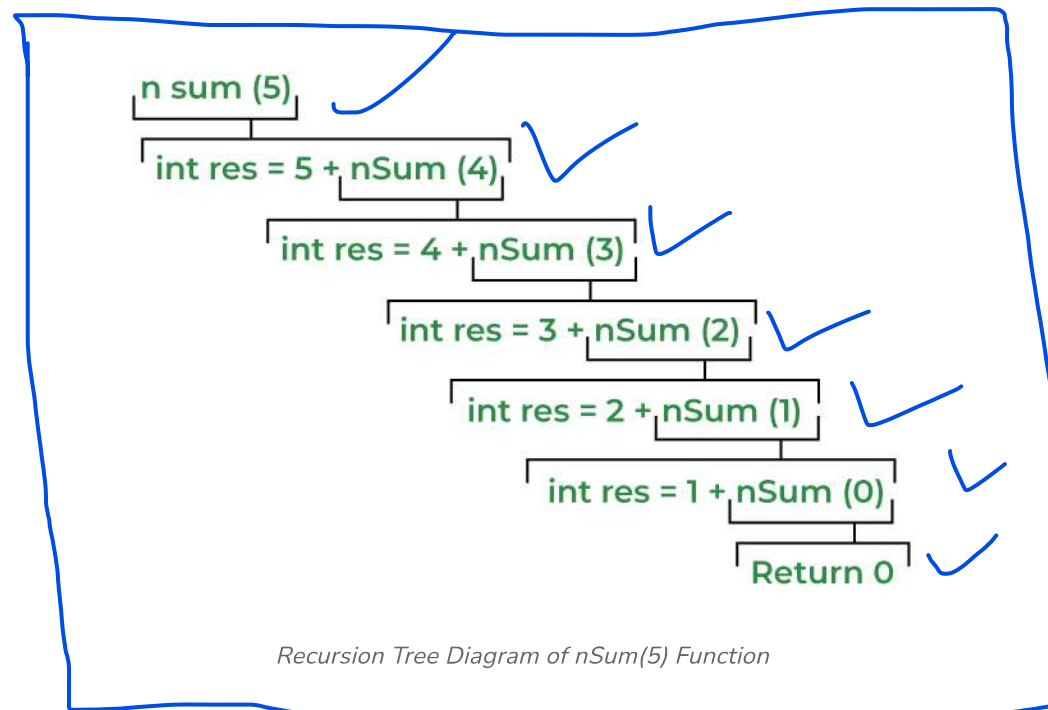
In nSum() function, the **base condition** is

```
if (n == 0) {  
    ... return 0;  
}
```

which means that when nSum(0) will return 0. Putting this value in nSum(5)'s recursive case, we get

```
int res = 5 + 4 + 3 + 2 + 1 + 0 = 15
```


At this point, we can see that there are no function calls left. So the recursion will stop here and the final value returned by the function will be **15** which is the sum of the first 5 natural numbers.



If you didn't understand properly and are still confused about the recursion working, don't worry, it is explained again in terms of memory management and the compiler's internal handling.

Memory Allocation for C Recursive Function

To further improve our understanding of recursion in C, we will look into how the recursion is internally handled by the C compiler and how the memory is managed for recursive functions.

As you may know, all the function's local variables and other stuff are stored inside the **stack frame** in stack memory and once the function returns some value, its stack frame is removed from the memory. The recursion follows a similar concept but with a little twist. In Recursion,

- *A stack frame is created on top of the existing stack frames each time a recursive call is encountered and the data of each recursive copy of the function will be stored in their respective stack.*
- *Once, some value is returned by the function, its stack frame will be destroyed.*
- *The compiler maintains an instruction pointer to store the address of the point where the control should return in the function after its progressive copy returns some value. This return point is the statement just after the recursive call.*
- *After all the recursive copy returned some value, we come back to the base function and the finally return the control to the caller function.*

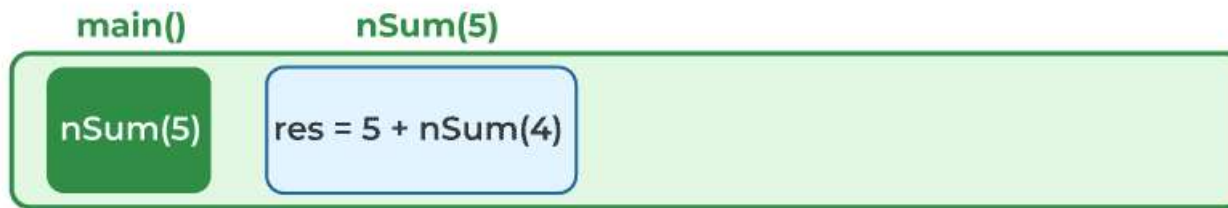
Let's use the first example again and see how the memory is managed for the nSum() function.

Step 1:

When nSum() is called from the main() function with 5 as an argument, **a stack frame for nSum(5)** is created.

Step 2:

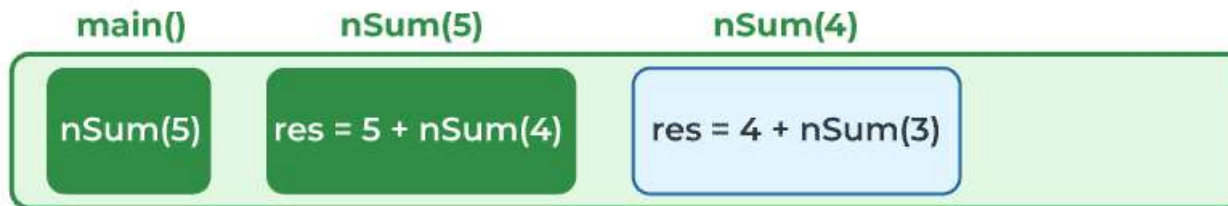
While executing nSum(5), a **recursive call** is encountered as **nSum(4)**. The compiler will now create a new stack frame on top of the nSum(5)'s stack frame and maintain an instruction pointer at the statement where **nSum(4)** was encountered.



Function Call Stack at the Execution of nSum(5)

Step 3:

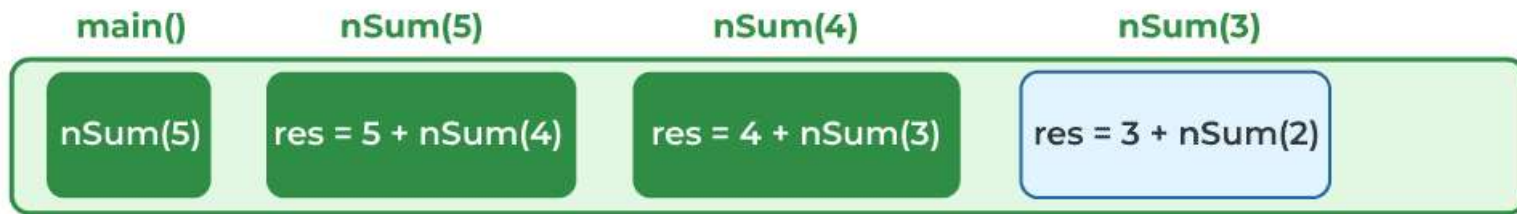
In the execution of **nSum(4)**, we encounter another recursive call as **nSum(3)**. The compiler will again follow the same steps and maintain another instruction pointer and stack frame for nSum(3).



Function Call Stack at the Execution of nSum(4)

Step 4:

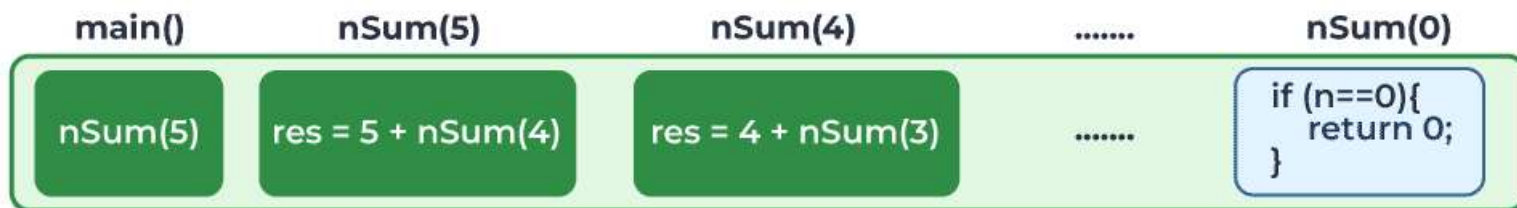
The same thing will happen with nSum(3), nSum(2), and nSum(1)'s execution.



Function Call Stack at the Execution of `nSum(3)`

Step 5:

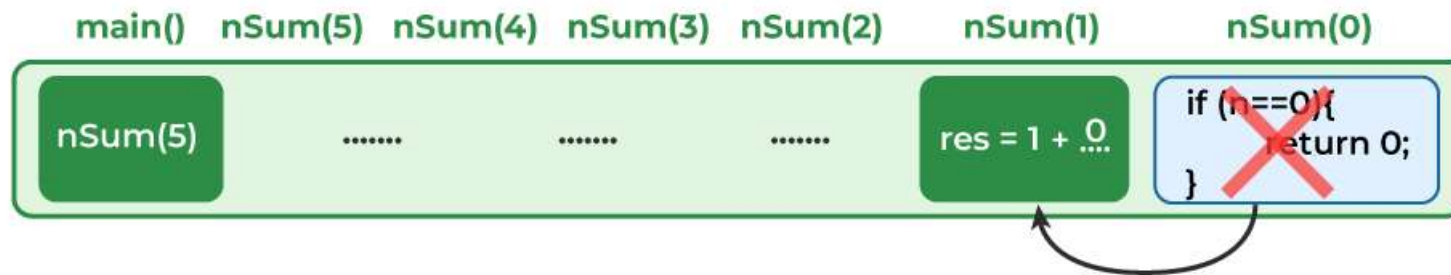
But when the control comes to `nSum(0)`, the condition `(n == 0)` becomes true and the statement `return 0` is executed.



Function Call Stack at the Execution of `nSum(0)`

Step 6:

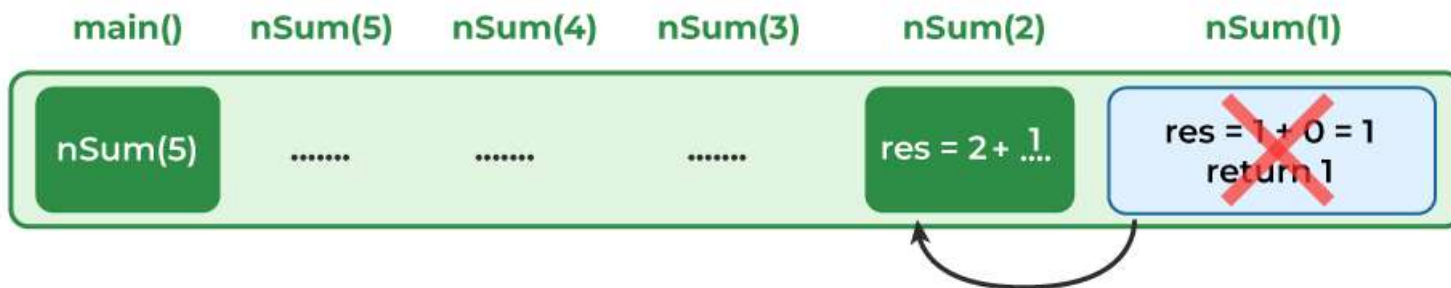
As the value is returned by the `nSum(0)`, the stack frame for the `nSum(0)` will be destroyed. Using the instruction pointer, the program control will return to the `nSum(1)` function and the `nSum(0)` call will be replaced by value 0.



nSum(0) Function Returning Value

Step 7:

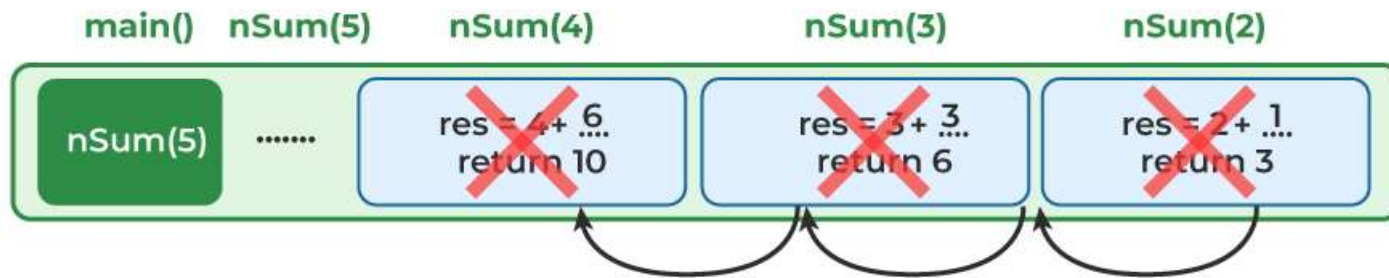
Now, in `nSum(1)`, the expression `int res = 1 + 0` will be evaluated and the statement `return res` will be executed. The program control will move to the `nSum(2)`.



nSum(1) Function Returning Value

Step 8:

In `nSum(2)`, `nSum(1)` call will be replaced by the value it returned, which is 1. So, after evaluating `int res = 2 + 1`, 3 will be returned to `nSum(3)`. The same thing will keep happening till the control comes to the `nSum(5)` again.



nSum(2), nSum(3) and nSum(4) Functions Returning Value

Step 9:

When the control reaches the `nSum(5)`, the expression `int res = 5 + nSum(4)` will look like `int res = 5 + 10`. Finally, this value will be returned to the `main()` function and the execution of `nSum()` function will be completed.



Output = 15

Final Result Returned to main()

Stack Overflow

The program's call stack has limited memory assigned to it by the operating system and is generally enough for program execution. But if we have a recursive function that goes on for infinite times, sooner or later, the memory will be exhausted and no more data can be stored. This is called stack overflow. In other words,

Stack overflow is the error that occurs occurs when the call stack of the program cannot store more data resulting in program termination.

It is one of the most common errors that is associated with the recursion.

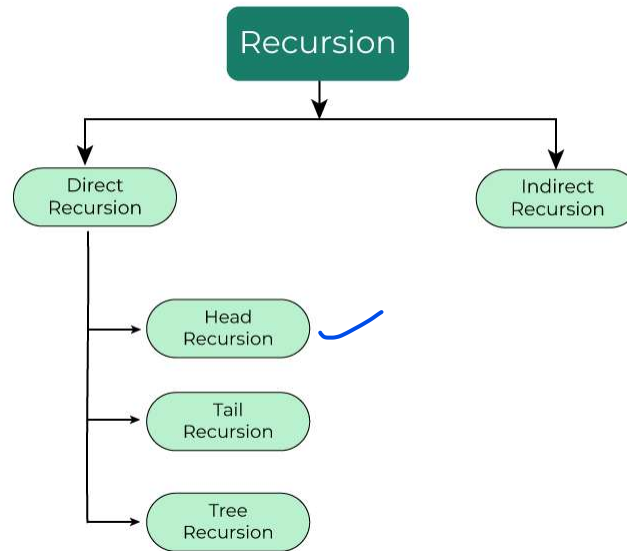
Types of C Recursion

In C, recursion can be classified into different types based on what kind of recursive case is present. These types are:

1. Direct Recursion

- Head Recursion
 - Tail Recursion
 - Tree Recursion
- 

2. Indirect Recursion



Types of Recursion in C

1. Direct Recursion

Direct recursion is the most common type of recursion, where a function calls itself directly within its own body. The recursive call can occur once or multiple times within the function due to which we can further classify the direct recursion

A. Head Recursion

The head recursion is a linear recursion where the position of its only recursive call is at the start of the function. It is generally the first statement in the function.

B. Tail Recursion

The tail recursion is also a liner recursion like head recursion but the position of the recursive call is at the end of the function. Due to this, the tail recursion can be optimized to minimize the stack

memory usage. This process is called Tail Call Optimization.

In the first example, the nSum() does the tail recursion.

C. Tree Recursion

In tree recursion, there are multiple recursive calls present in the body of the function. Due to this, while tracing the program flow, it makes a tree-like structure, hence the name Tree Recursion.

2. Indirect Recursion

Indirect recursion is an interesting form of recursion where a function calls another function, which eventually calls the first function or any other function in the chain, leading to a cycle of function calls. In other words, the functions are mutually recursive. This type of recursion involves multiple functions collaborating to solve a problem.

Examples of Recursion in C

Example 1: C Program to Find the Factorial of a Natural Number using Tail Recursion.

C

```
// C program to find the factorial using tail recursion
#include <stdio.h>

int factorialTail(int n)
{
    // Base case
    if (n == 1 || n == 0) {
        return 1;
    }
}
```

```

..... else {
.....     // Tail recursive call
.....     return n * factorialTail(n - 1);
..... }
}

int main()
{

..... int n = 5;

..... int fact1 = factorialTail(n);
..... printf("Resursive Factorial of %d: %d \n", n, fact1);

..... return 0;
}

```

Output

Resursive Factorial of 5: 120

Example 2: C Program to find the Fibonacci Number using Tree Recursion

C



```

// C Program to find the fibonacci number using tree
// recursion

```



```
#include <stdio.h>

int fibonacci(int n)
{
    .... // Base case
    .... // Fibonacci of 0 and 1 is the number itself
    .... if (n <= 1) {
    ....     return n;
    .... }
    .... else {
    ....     // Tree recursive calls
    ....     return fibonacci(n - 1) + fibonacci(n - 2);
    .... }
}

int main()
{
    .... // function call
    .... int n = fibonacci(3);

    .... // print 5th fibonacci number
    .... printf("%d", n);

    .... }
```



Output

2

Example 3: C Program to Illustrate the Indirect Recursion

C



```
// C Program to Illustrate the Indirect Recursion
#include <stdio.h>

void functionA(int n)
{
    if (n < 1) {
        return;
    }
    printf("%d ", n);
    n = n - 1;

    // Indirect recursive call to functionB
    functionB(n);
}

void functionB(int n)
{
    if (n < 2) {
        return;
    }

    printf("%d ", n);
    n = n / 2;

    // Indirect recursive call to functionA
    functionA(n);
}
```

Output

20 10 9 4 3 1

Applications of Recursion in C

Recursion is widely used to solve different kinds of problems from simple ones like printing linked lists to being extensively used in AI. Some of the common uses of recursion are:

- Tree-Graph Algorithms
- Mathematical Problems
- Divide and Conquer
- Dynamic Programming
- In Postfix to Infix Conversion
- Searching and Sorting Algorithms

Advantages of C Recursion

The advantages of using recursive methods over other methods are:

1. Recursion can effectively reduce the length of the code.
2. Some problems are easily solved by using recursion like the tower of Hanoi and tree traversals.
3. Data structures like linked lists, trees, etc. are recursive by nature so recursive methods are easier to implement for these data structures.

Disadvantages of C Recursion

As with almost anything in the world, recursion also comes with certain limitations some of which are:

1. Recursive functions make our program a bit slower due to function call overhead.

2. Recursion functions always take extra space in the function call stack due to separate stack frames.
3. Recursion methods are difficult to understand and implement.

Conclusion

It is said that,

Any problem that can be solved using iteration can also be solved with recursion and vice versa.

So, C recursion is just one of the methods to solve problems of different kinds. It provides a lot of benefits but also comes with certain limitations. It should be used when its benefits outweigh its limitations.

FAQs on C Recursion

Q1. What is the difference between iteration and recursion in C?

Answer:

The following table lists the differences between recursion and iteration.

<i>Property</i>	<i>Iteration</i>	<i>Recursion</i>
<i>Definition</i>	<i>A set of instructions repeatedly executed.</i>	<i>Function calls itself.</i>
<i>Termination</i>	<i>When the termination condition for the iterator ceases to be satisfied.</i>	<i>Through base case, where there will be no function call.</i>
<i>Usage</i>	<i>Used when code size needs to be small, and time complexity is not an issue.</i>	<i>Used when time complexity needs to be balanced against an expanded code size.</i>
<i>Time Copmlexity</i>	<i>Very high(generally exponential) time complexity.</i>	<i>Relatively lower time complexity(generally polynomial-logarithmic).</i>
<i>Space Complexity</i>	<i>Space complexity is lower.</i>	<i>The space complexity is higher than iterations.</i>

To know more about the differences, refer to this article – [Difference between Iteration and Recursion](#)

Q2. What is tail call optimization?

Answer:

Tail call optimization refers to the optimizing method applicable to tail recursion in which the return value is stored by the compiler and the same stack frame is used for each recursive call instead of separator memory.

To know more about it, refer to this article – [Tail Call Elimination](#)

Three 90 Challenge is back on popular demand! After processing refunds worth INR 1CR+, we are back with the offer if you missed it the first time. Get 90% course fee refund in 90 days. [Avail now!](#)

"The DSA course helped me a lot in clearing the interview rounds. It was really very helpful in setting a strong foundation for my problem-solving skills. Really a great investment, the passion Sandeep sir has towards DSA/teaching is what made the huge difference." - **Gaurav | Placed at Amazon**

Before you move on to the world of development, **master the fundamentals of DSA** on which every advanced algorithm is built upon. Choose your preferred language and start learning today:

[DSA In JAVA/C++](#)

[DSA In Python](#)

[DSA In JavaScript](#)

Trusted by Millions, Taught by One- Join the best DSA Course Today!